3.1 JavaScript execution and scope

A deep understanding of JavaScript's execution model is essential for excelling in technical interviews. Topics like scope, hoisting, closures, and the often-confusing behavior of this frequently trip up candidates—but they are foundational to reasoning about how JavaScript code runs, how variables are resolved, and how context is maintained at runtime.

Interviewers use these concepts to assess not only theoretical knowledge but also a developer's ability to debug and analyze real-world code. Whether it is explaining lexical scoping, tracing the scope chain, or resolving a tricky `this` binding in an event handler, these skills underpin countless front-end and full-stack interview challenges.

Table 3.1 serves as Jane's review guide for execution and scope. Use it as a checklist to ensure you have mastered these core concepts before stepping into any technical interview.

Table 3.1 Core concepts of execution and scope

Topic	Fundamentals	Applied skills	Advanced
Execution context & scope	Define global/function/block scope, explain variable hoisting.	Trace scope chain; explain closures.	Debug closure bugs, analyze memory leaks; optimize lexical scoping.
The `this` keyword	Explain `this` in functions/objects, show basic usage.	Use `bind`, `call`, `apply`; handle `this` in arrow functions and events.	Solve tricky `this` bugs; manage context in frameworks; design mindful APIs.
Type coercion & equality	Explain == vs ===, identify truthy/falsy values.	Prevent coercion bugs; handle edge cases; use explicit	Audit coercion issues, enforce linting rules;